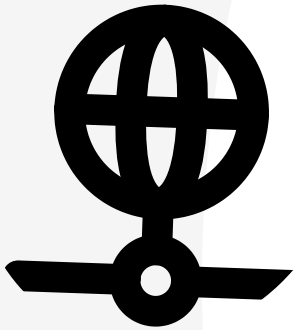
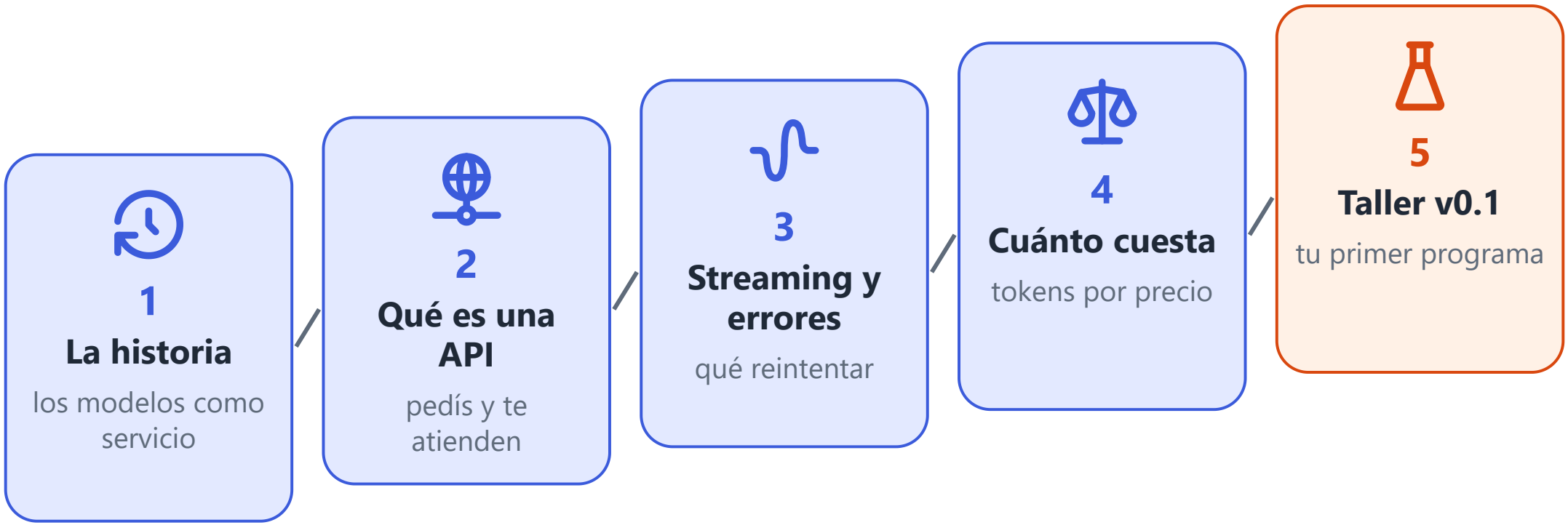
# APIs

## M3: APIs, v0.1



Hoy: el recorrido

# Del chat a un programa que mide




Venís de pedirle datos con un molde. Ahora, desde tu propio programa.

# Sin mirar: ¿qué no detectó tu molde?

?

**Tu error de M2**

uno que el molde dejó pasar: el dato entró, pero estaba mal

**Escribilo en una línea: lo vamos a necesitar al final.**

¿Qué error dejó pasar tu molde?

# Lo que el molde suele dejar pasar



**1,25 por 1.250**

una cifra mal leída que igual es un número



**999**

la orden escondida del texto 7: también es un número válido

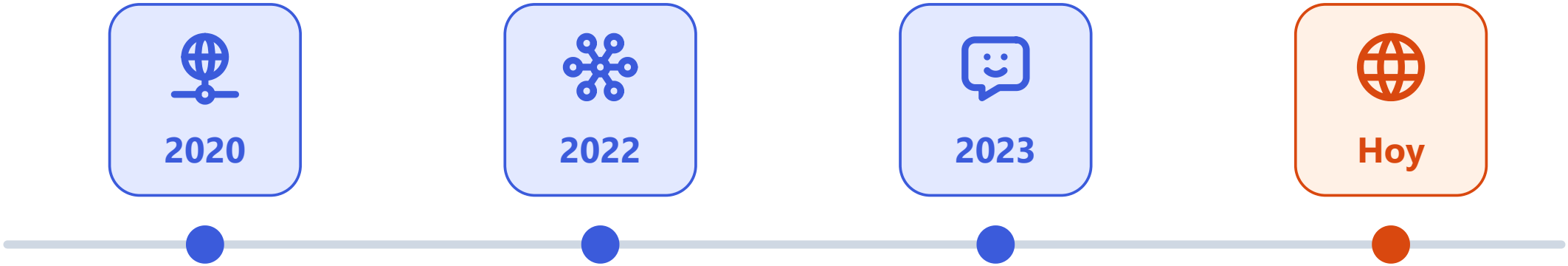


**«Las Acacias»**

una entidad incompleta: forma bien, dato mal

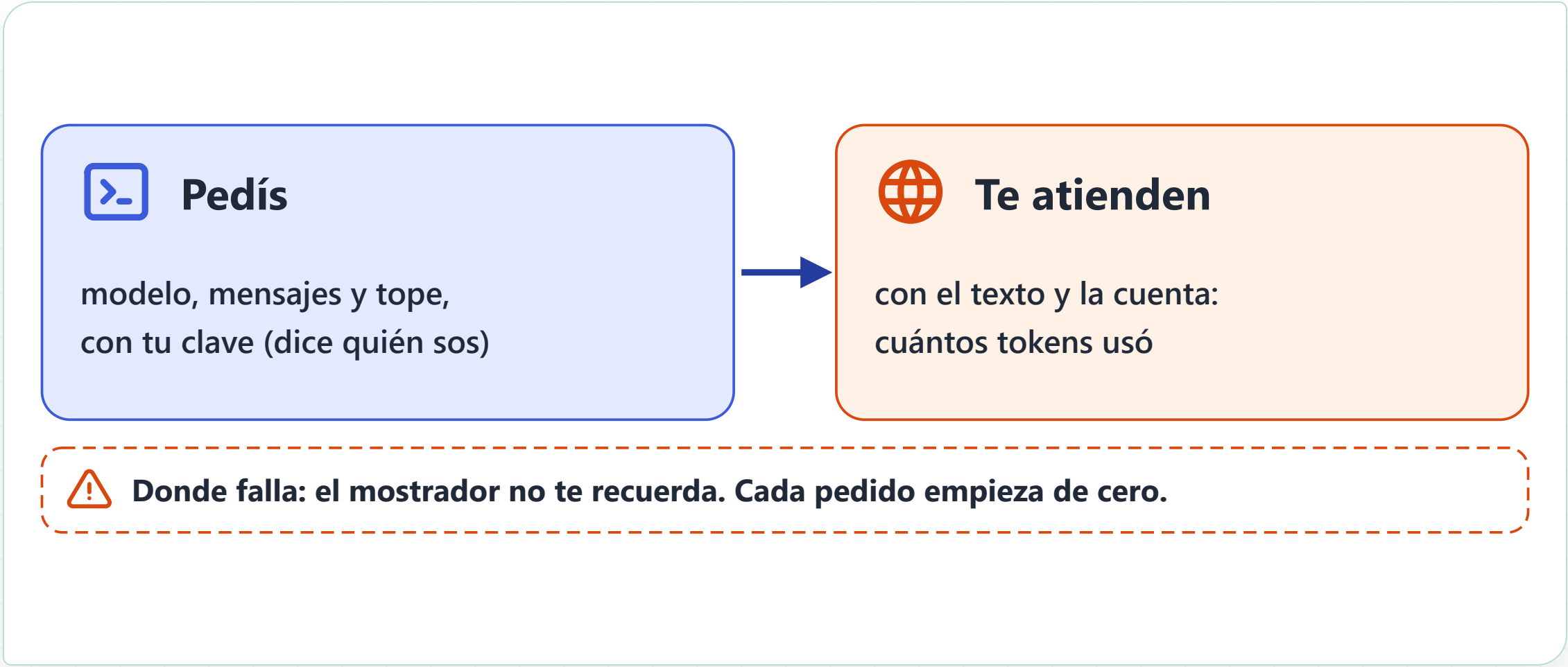
**El molde controla la forma, no la verdad.**

# Los modelos se usan como un servicio



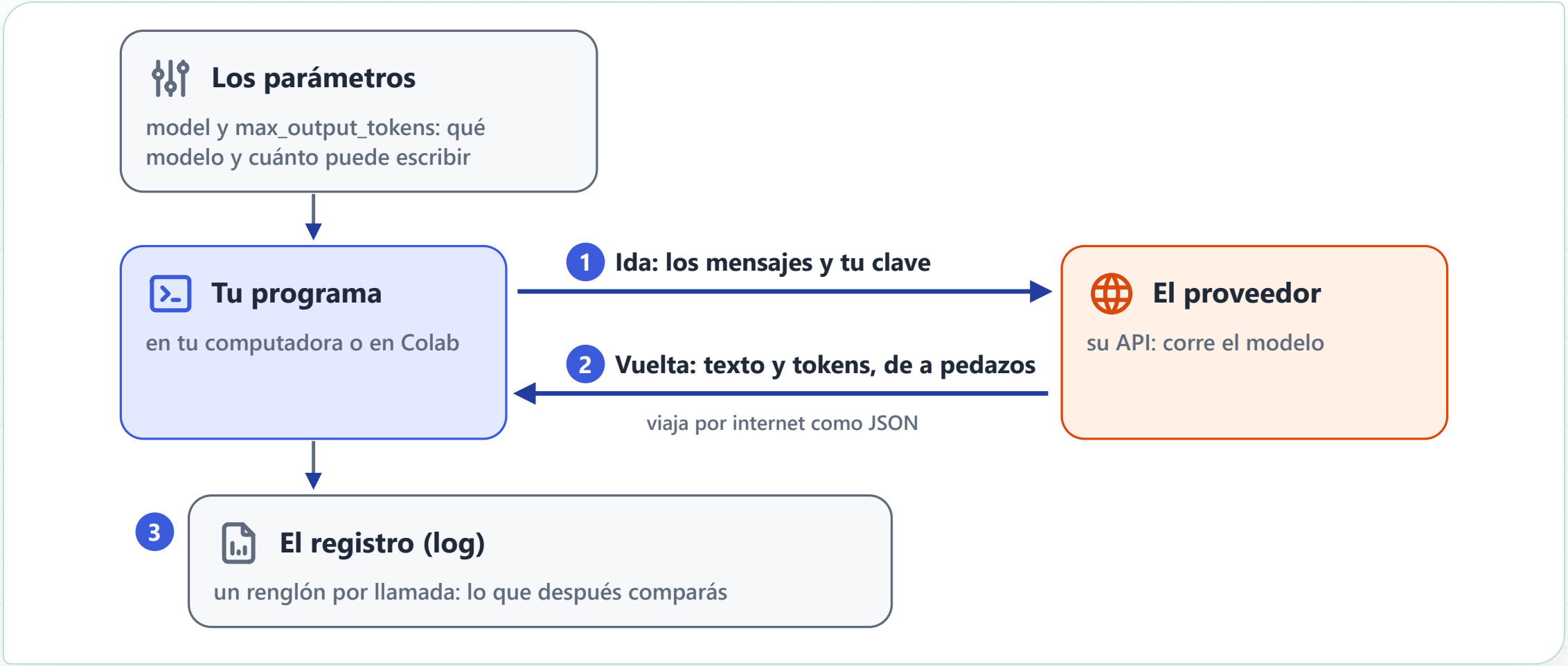

**Por eso tu código cambia poco si cambiás de proveedor.**

# Una API es un mostrador: pedís y te atienden




Como Power BI contra tu base: credencial, consulta y resultado.

# Una llamada es un pedido que viaja y vuelve




Va el pedido con tu clave; vuelven texto y tokens.

# Lo que viaja es una lista de mensajes con rol

```
[
 {"role": "system",
   "content": "Sos un asistente para estudiantes…"},
 {"role": "user",
   "content": "¿Qué es un token en un LLM?"}
]
```

**system: las reglas de siempre**

**user: la pregunta de ahora**

Viaja la lista entera en cada llamada.

# Cada mensaje es un par (rol, texto)

```
[("system", SISTEMA), ("human", pregunta)]
```

lo que escribís

| rol | texto | qué es |
|---|---|---|
| system | «Sos un asistente para estudiantes…» | las reglas de siempre |
| human | «¿Qué es un token en un LLM?» | la pregunta de ahora |

**Vuelve:** el texto · los tokens · los segundos

Una tabla de dos columnas: rol y texto.

# Lo que vuelve: el texto y la cuenta de tokens

**Ollama, qwen3.5:2b**

Sistema: Sos un asistente para estudiantes. Respondé en español, en dos frases…

¿Qué es un token en un LLM?

Un token es la unidad mínima de texto que procesa un modelo de lenguaje artificial, ya sea una palabra completa o una parte de ella…

**Vuelve también:**

**45 de entrada**
con el sistema incluido

**52 de salida**

**97 en total**

Se pagan.

# El modelo no recuerda: la historia la mandás vos

**A · solo la pregunta**

¿Cómo me llamo?

No tengo nombre propio, soy una inteligencia artificial generada por humanos. ¿En qué puedo ayudarte hoy?

**B · con la historia**

Me llamo Ana.

¡Hola, Ana!

¿Cómo me llamo?

Tu nombre es Ana, y yo soy tu asistente virtual.

Mismo modelo y sistema.

# Tres palabras cambiaron la respuesta correcta

**A · ambigua**

¿Qué es una alucinación?

Una alucinación es una percepción falsa que ocurre cuando el cerebro interpreta estímulos no reales como si fueran verdaderos.

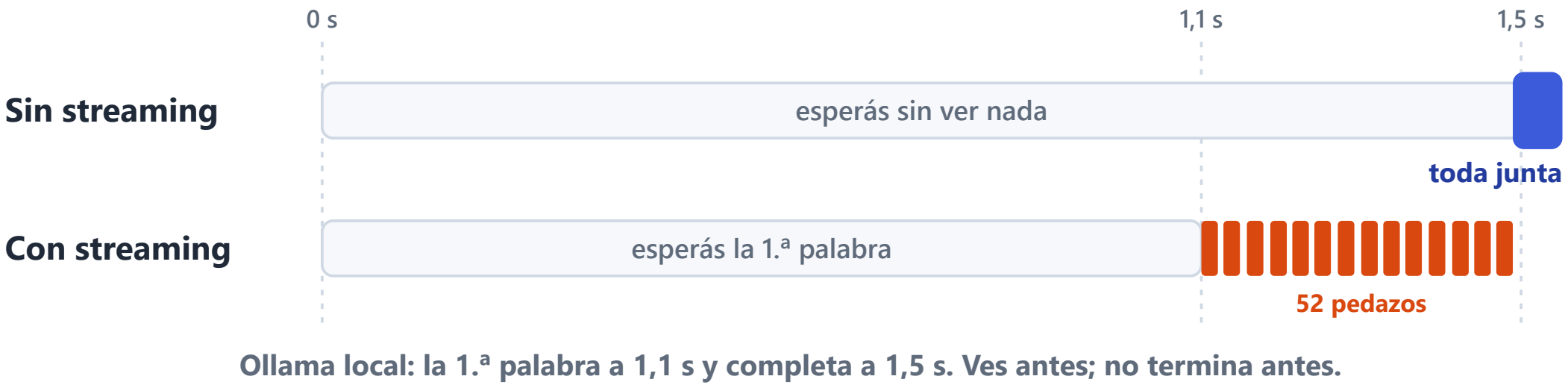
**B · con «en un LLM»**

¿Qué es una alucinación en un LLM?

Una alucinación en un modelo de lenguaje artificial ocurre cuando el sistema genera información falsa o inventada…

Medí con preguntas bien escritas.

# El streaming muestra antes, no termina antes




**Ollama local: la 1.ª palabra a 1,1 s y completa a 1,5 s. Ves antes; no termina antes.**

**Streaming** = la respuesta llega en pedazos.

# El streaming junta pedazos para medir

```
def preguntar(modelo, pregunta):
    inicio, respuesta = time.perf_counter(), None
    for pedazo in modelo.stream(
            [("system", SISTEMA), ("human", pregunta)]):
        print(pedazo.text, end="", flush=True)
        respuesta = pedazo if respuesta is None else respuesta + pedazo
    print()
    return respuesta.text, respuesta.usage_metadata, \
        round(time.perf_counter() - inicio, 2)
```

**Arranca el reloj**

**Los pedazos llegan de a uno**

**Los suma: al final queda la respuesta entera**

**Vuelven texto, tokens y segundos**

Cada pedazo se muestra apenas llega.

# En la nube, cada pedazo trae más tokens

**Ollama: los primeros pedazos**

`Un` `·token` `·es` `·la` `·unidad` `·mínima` …

52 pedazos · 1.ª palabra a 1,1 s · completa a 1,5 s

**Gemini: 5 o 6 pedazos por respuesta**

`Un` | una frase entera | una frase entera | una frase entera | una frase entera

El tamaño de cada pedazo lo decide el proveedor.

# El tope de salida corta la respuesta

**Con un tope de 12 tokens de salida**

¿Para qué sirve la temperatura en un modelo de lenguaje?

La temperatura controla el equilibrio entre la precisión y la creatividad

sin punto final

tokens de entrada: 47

**tokens de salida: 12**

motivo de fin:

`length (se cortó)`

En la API: `max_output_tokens` (Gemini) o `max_tokens` (OpenAI, Claude).

# Cada respuesta HTTP trae un código

| Código | Qué dice | Qué hacés |
|---|---|---|
| **200** | Salió bien | Leés la respuesta |
| **400, 401, 404** | Tu pedido está mal: formato, clave o modelo | Corregís |
| **429** | Demasiados pedidos | Esperás y reintentás |
| **500, 503** | Falló el proveedor | Reintentás |

# Reintentá lo pasajero, no lo que está mal

## Reintentar

pasajero: esperá y otra vez

| | |
|---|---|
| `429` | demasiados pedidos |
| `timeout` | tardó demasiado |
| `5xx` | falla del servidor |

## Corregir

está mal: reintentar no sirve

| | |
|---|---|
| `400` | pedido mal armado |
| `401` | clave inválida (Gemini: 400) |
| `404` | modelo inexistente |

Como en Power BI: un corte se reintenta; una credencial vencida, no.

# Un 429 por día no se arregla reintentando

```
429 RESOURCE_EXHAUSTED.
You exceeded your current quota …
Quota exceeded for metric:
  generate_content_free_tier_requests,
  limit: 500, model: gemini-3.5-flash-lite
Please retry in 7h32m8s.
```

— **Cuota gratis**

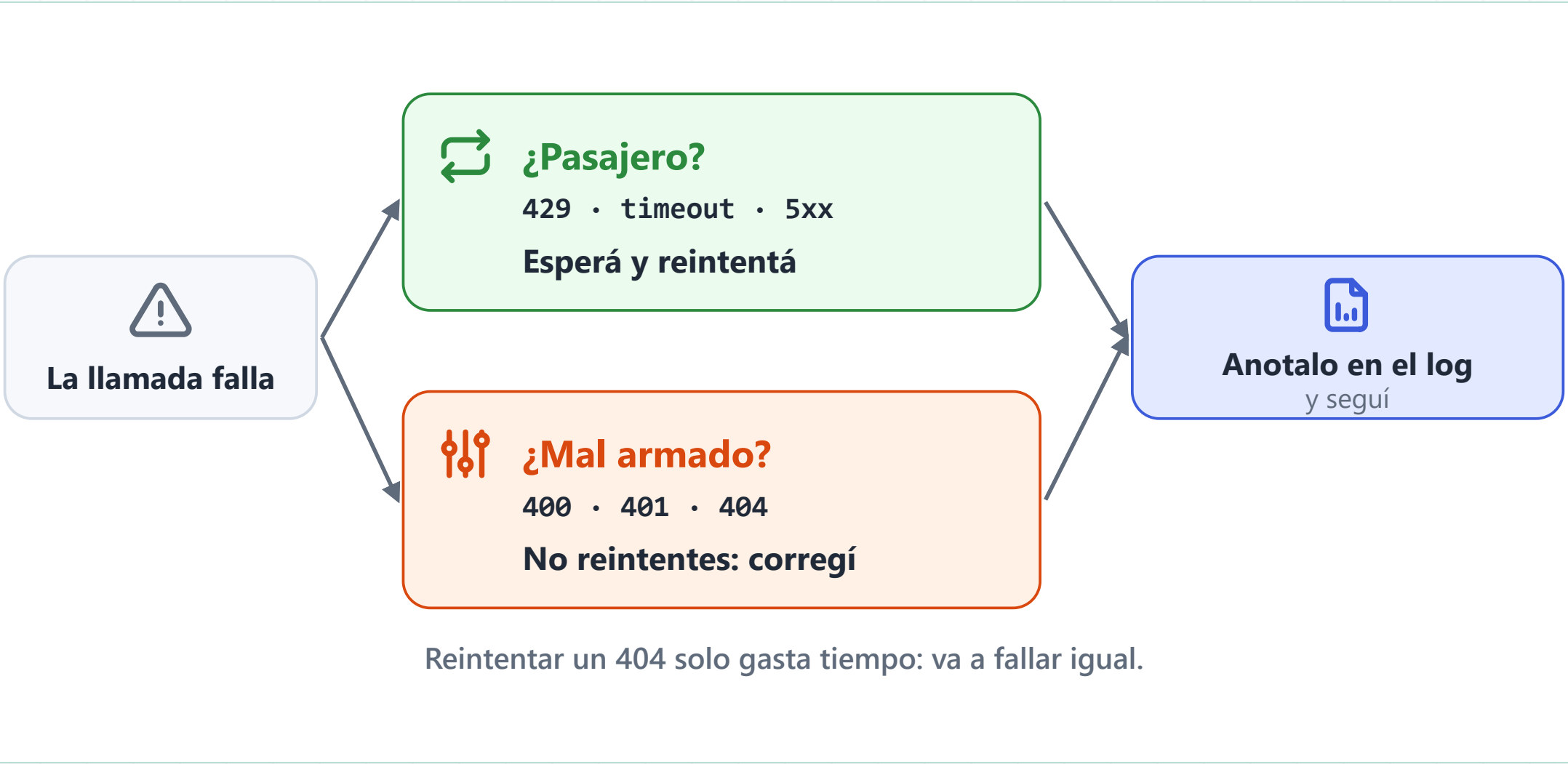
— **500 pedidos por día y por modelo**

— **Pide esperar horas: es la cuota del día**

**No reintentes: esperá a mañana o cambiá de modelo.**

¿Pide esperar horas o dice «PerDay»? No reintentes.

# Ante un error, decidís si esperar o anotar




Reintentar un 404 solo gasta tiempo: va a fallar igual.

# Reintentar: otra vez, con una espera

```
for intento in range(1, REINTENTOS + 1):
    try:
        texto, uso, segundos = preguntar(...)
        registro.append({...})
        break
    except Exception as error:
        if intento < REINTENTOS:
            time.sleep(2 * intento)
        else:
            registro.append({"error": ...})
```

- Hasta REINTENTOS veces
- Salió: no reintenta
- 429, 503, red: falló esta vez
- Espera 2 s, 4 s… antes de otra vez
- Se acabaron: lo anota y sigue

Si sigue fallando, se anota y sigue.

# El error a propósito queda anotado

m3_v01_api.ipynb

```python
try:
    init_chat_model("google_genai:modelo-que-no-existe").invoke("hola")
except Exception as error:
    registro.append({"modelo": "google_genai:modelo-que-no-existe", "error": type(error).__name__})
    print("Error controlado:", type(error).__name__)
```

Un 404 no se reintenta.

# Otro proveedor: otros tokens, otro error

| | Ollama | Gemini 3.5 Flash-Lite |
|---|---|---|
| Tokens de entrada | 45 | 28 |
| Tiempo | 15 s la 1.ª, 0,3 s luego | 1 s, a veces 20 s |
| Error | `ResponseError` | `GoogleModelNotFoundError` |

Otro tokenizador: tokens no comparables.

# Predecí qué pasa en tres llamadas

1. **Predecí** y anotá las tres respuestas.
2. **Mirá** las tres corridas en vivo: `memoria`, `pedazos` y `tope`.
3. **Compará** con tu predicción.



**`memoria`** ?

¿Recuerda tu nombre sin la historia?



**`pedazos`** ?

Corré 2 veces: ¿cuándo llega la 1.ª palabra?



**`tope`** ?

¿Con qué motivo de fin termina?

# Cada pregunta deja un renglón en el log

```
{"modelo": "ollama:qwen3.5:2b",
  "sistema": "Sos un asistente…",
  "pregunta": "¿Qué es una alucinación…?",
  "respuesta": "Una alucinación en…",
  "latencia_s": 0.32,
  "tokens_entrada": 47,
  "tokens_salida": 30}
```

— **Con qué modelo**

— **Lo que devolvió el modelo**

— **Cuánto tardó, en segundos**

— **Tokens que entran y salen: de acá sale el costo**

**Cada llamada agrega un renglón; no borra.**

JSONL: un diccionario por renglón, como una fila de tabla.

# LangSmith muestra cada llamada en la web

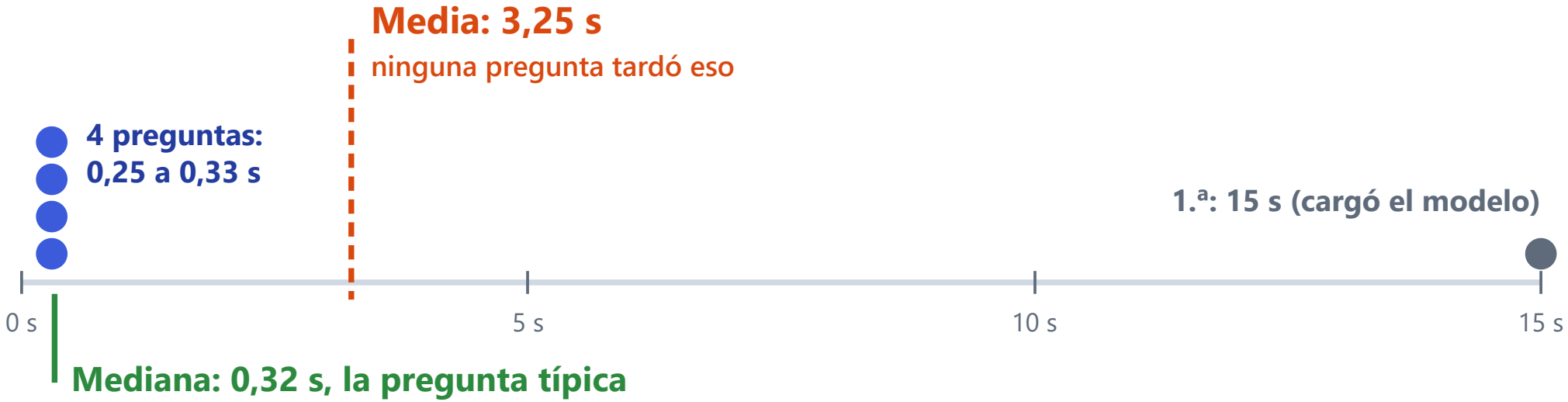
**Tu registro**
un renglón por llamada, lo armás vos

→

**LangSmith**
mensajes, respuesta, tokens y tiempo, en la web

Sin cambiar código: tres variables (`LANGSMITH_TRACING`, `LANGSMITH_API_KEY`, `LANGSMITH_PROJECT`).

# La media engaña: una llamada lenta la arrastra




Ollama, 17/10: la primera tardó 15 s porque cargó el modelo.

# ¿Qué pesa más en el costo de tu consulta?

**Costo = tokens de entrada × su precio**
**+ tokens de salida × su precio**

**Entrada**

**500 tokens**

US$ 0,10 por millón

**Salida**

**200 tokens**

US$ 0,40 por millón

**Total del ejemplo: US$ 0,00013. ¿Qué pesa más: la entrada o la salida?**

¿La entrada o la salida?

# La salida pesa más, aunque sean menos tokens




Con estos precios, cada token de salida cuesta 4 veces más. En Gemini 3.5 Flash-Lite (19/10), 8.

Mirá la salida: es la que más cuesta.

# El costo sale de los tokens

```
PRECIO_ENTRADA = 0.30
PRECIO_SALIDA = 2.50
CONSULTAS_POR_DIA = 1000
por_pregunta = bien["entrada"].mean() * PRECIO_ENTRADA / 1_000_000 \
             + bien["salida"].mean() * PRECIO_SALIDA / 1_000_000
print(... por_pregunta * CONSULTAS_POR_DIA * 30 ...)
```

— 8 veces la de entrada

— la medida: tokens × precio ÷ millón

— × 1000 por día × 30 días: US$ 3,75 por mes (21/10)

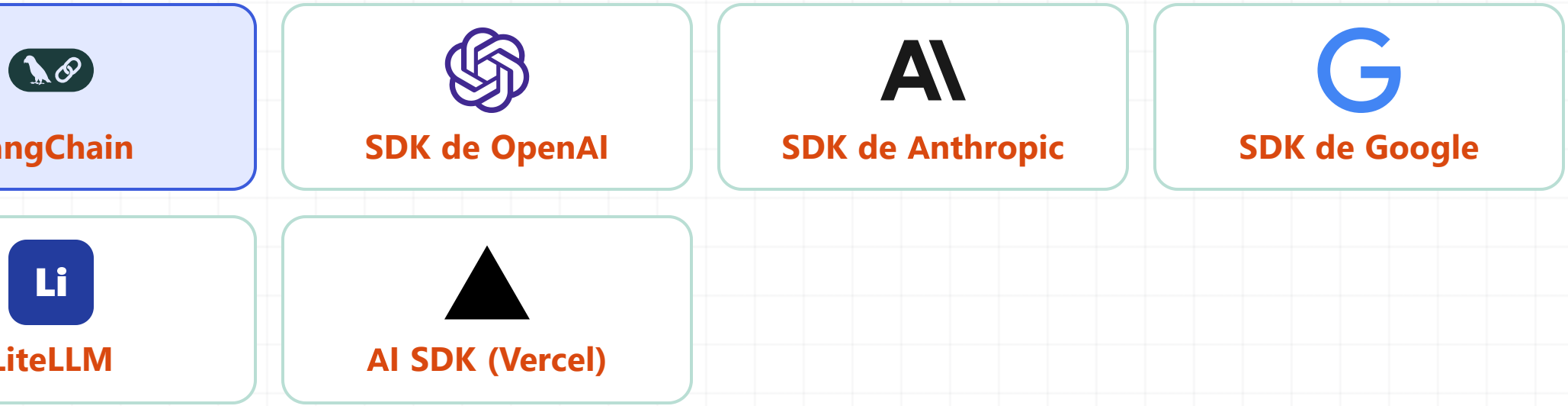
Una medida, como en Power BI: US$ 3,75 por mes (21/10).

# Cambiar de proveedor es una línea, casi

| | Gemini 3.5 Flash-Lite | GPT-5.4 nano | Claude Haiku 5.5 |
|---|---|---|---|
| Usa temperatura | ✗ | ✗ | ✗ |
| Usa herramientas | ✓ | ✓ * | ✓ |

* 12 de 12 en selección. «Casi»: cambia el nombre del modelo (model) y la clave del proveedor. Probado del 15 al 17/10.

# LangChain no es la única forma de llamarlos




Cada proveedor tiene su **SDK**, su librería para llamarlo. LangChain los unifica: cambiar es una línea.

# Taller: tus preguntas, dos configuraciones (25 min)

1. **Escribí** `responder()` (práctica M3).
2. **Tu tema:** el sistema y 5 preguntas tuyas.
3. **Dos configuraciones:** predecí y compará, también en LangSmith.




**Primer control · 10–15 min:** `logs/v01.jsonl` con tokens.

# Repaso: hoy te llevás tres ideas



### API

mandás mensajes con tu clave, vuelven texto y tokens



### Errores

reintentá lo pasajero (429, 5xx), corregí lo que está mal



### Costo

sale de los tokens, y la salida pesa más

# Cerrá v0.1: commit y una decisión

Entrega de avance 1: `evidencia/v0.1.md`, *commit* y *push* hasta el mié 4/11, 23:59.

¿Qué error reintentás y cuál no? ¿Por qué?

Anotá tu **confianza (1 a 4)**. El mié, **Control de lectura 1** sobre las lecturas del Módulo 4 (en el aula virtual).